# RepMateで学ぶ TypeScript・CRUD 必修ガイド

対象: TypeScript / React / Next.js App Router / Prisma / Supabase Auth を学習中の方向け
題材: `/report` のトレーニング記録機能

---

## 1. この資料のゴール

まず、次の一文を自分の言葉で説明できることを目標にしてください。

> 画面から `fetch` でAPIを呼び、APIで認証と入力検証を行い、PrismaでDBを操作し、結果をJSONで画面へ返す。

最低限、次の5点は必ず身につけたい内容です。

1. CRUDとHTTPメソッドの対応
2. `async / await` を使った非同期処理
3. TypeScriptの型と `unknown` の意味
4. Zodによる実行時の入力検証
5. PrismaによるDB操作と、ログインユーザー本人のデータだけを扱う方法

この5点を理解すると、Todoアプリ、日報、投稿、予約、プロフィール編集などにも応用できます。

---

## 2. RepMateのCRUD対応表

CRUDはデータ操作の4種類をまとめた呼び方です。

| CRUD | 意味 | HTTP | RepMateの処理 | Prisma |
|---|---|---|---|---|
| Create | 新規作成 | `POST` | トレーニング記録を保存 | `createMany()` |
| Read | 取得 | `GET` | 本日の記録を一覧表示 | `findMany()` |
| Update | 更新 | `PATCH` | 重量・回数を編集 | `update()` |
| Delete | 削除 | `DELETE` | 記録を削除 | `deleteMany()` |

APIのURLは次の2種類です。

```
/api/training-records       GET / POST
/api/training-records/[id]  PATCH / DELETE
```

`[id]` は動的ルートです。例えば、IDが `abc` なら次のURLになります。

```
/api/training-records/abc
```

---

## 3. 最初に覚える全体の流れ

```
ユーザー操作
  ↓
ReportScreenのイベント関数
  ↓ fetch
Next.js Route Handler
  ↓
Supabase Authでログイン確認
  ↓
Zodでリクエスト内容を検証
  ↓
PrismaでPostgreSQLを操作
  ↓
NextResponse.json()で結果を返す
  ↓
画面のstateを更新して再表示
```

## requestとresponseをファイル単位で追う図

新規作成（POST）を例にすると、実際には次の2ファイル間で通信しています。

```
【画面・クライアント側】
src/app/(app)/report/_components/ReportScreen.tsx

saveRecord()
  |
  |  fetch("/api/training-records", { method: "POST", body: ... })
  |  ここから「HTTPリクエスト」を送信
  v
============================================================ HTTP
  v
【API・サーバー側】
src/app/api/training-records/route.ts

POST(request: Request)
  |  画面から送られたHTTPリクエストをrequestとして受け取る
  |  request.json()でbodyを取り出す
  |  Zodで検証する
  |  PrismaでDBへ保存する
  |
  |  NextResponse.json({ records }, { status: 201 })
  |  ここから「HTTPレスポンス」を返す
  v
============================================================ HTTP
  v
【画面・クライアント側】
src/app/(app)/report/_components/ReportScreen.tsx

const response = await fetch(...)
  |  APIから返されたHTTPレスポンスがresponseへ入る
  |
  +-- response.ok      成功したか
  +-- response.status  201などのステータス
  +-- response.json()  JSON本文を取り出す
```

ここで重要なのは、`response`という変数がAPI側で`request`へ変換されるわけではないことです。

```
画面からAPIへ向かう通信 = request
APIから画面へ戻る通信   = response
```

変数名は任意ですが、RepMateでは役割が分かりやすいように次の名前を使っています。

```
// API側: 受け取ったリクエスト
export const POST = async (request: Request) => {
  const body = await request.json();
};

// 画面側: APIから返されたレスポンス
const response = await fetch("/api/training-records");
const data = await response.json();
```

## CRUDごとの送信元と受信先

| 処理 | 画面側の関数 | HTTP | API側のファイル・関数 |
| --- | --- | --- | --- |
| 一覧取得 | `fetchRecords()` | `GET` | `api/training-records/route.ts` の `GET()` |
| 新規作成 | `saveRecord()` | `POST` | `api/training-records/route.ts` の `POST()` |
| 更新 | `saveRecordEdit()` | `PATCH` | `api/training-records/[id]/route.ts` の `PATCH()` |
| 削除 | `deleteRecord()` | `DELETE` | `api/training-records/[id]/route.ts` の `DELETE()` |

`GET` と `POST` はIDを指定しないため通常の `route.ts` へ届きます。`PATCH` と `DELETE` は対象のIDをURLへ含めるため、`[id]/route.ts` へ届きます。

## 最重要: 覚えておくCRUDテンプレート早見表

この2つは、まず何度も書いて流れを覚えたいテンプレートです。後半の「16. 最低限、自力で書けるようにしたいテンプレート」にも、同じ内容と説明を残しています。

### 画面・クライアント側

```
const createItem = async () => {
  try {
    const response = await fetch("/api/items", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ name: "sample" }),
    });

    const data = await response.json();
    if (!response.ok) throw new Error(data.error);

    // 一覧を再取得する、またはstateへ追加する
  } catch (error) {
    console.error(error);
  }
};
```

### API・サーバー側

```
export const POST = async (request: Request) => {
  const body: unknown = await request.json().catch(() => null);
  const result = schema.safeParse(body);

  if (!result.success) {
    return NextResponse.json({ error: "入力エラー" }, { status: 400 });
  }

  const item = await prisma.item.create({
    data: result.data,
  });

  return NextResponse.json({ item }, { status: 201 });
};
```

```
画面: 送る → レスポンスを確認する → 画面を更新する
API : 受け取る → 検証する → DB操作する → 結果を返す
```

関連ファイルは次のとおりです。

| 役割 | ファイル |
| --- | --- |
| 画面・API呼び出し | `src/app/(app)/report/_components/ReportScreen.tsx` |
| 一覧取得・新規作成API | `src/app/api/training-records/route.ts` |
| 更新・削除API | `src/app/api/training-records/[id]/route.ts` |
| 入力検証 | `src/app/api/training-records/_schemas/training-record.ts` |
| レスポンスの型 | `src/lib/_types/training-record.ts` |
| DBモデル | `prisma/schema.prisma` |

---

## 4. DBモデルを読む

中心となるのは`TrainingRecord`モデルです。

```
model TrainingRecord {
  id         String   @id @default(uuid()) @db.Uuid
  userId     String   @map("user_id") @db.Uuid
  reportId   String?  @map("report_id") @db.Uuid
  exerciseId String   @map("exercise_id") @db.Uuid
  weight     Decimal  @db.Decimal(6, 2)
  reps       Int
  setOrder   Int      @map("set_order")
  memo       String?  @db.Text
  recordedAt DateTime @map("recorded_at") @db.Timestamptz(6)
}
```

### TypeScriptとの対応

| Prisma | TypeScriptで扱う値 | 意味 |
| --- | --- | --- |
| `String` | `string` | IDなどの文字列 |
| `Int` | `number` | 回数・セット順 |
| `Decimal` | Prismaの `Decimal` | 誤差を避けたい数値 |
| `String?` | `string \| null` | 値がない可能性がある |
| `DateTime` | `Date` | 日時 |

`?` は「省略可能」または「NULLを許容する」という意味です。DBの `memo String?` は、レスポンス型では `memo: string | null` になっています。

```
export type TrainingRecordItem = {
  id: string;
  exercise: string;
  weight: number;
  reps: number;
  setOrder: number;
  memo: string | null;
  recordedAt: string;
};
```

DBでは `recordedAt` が `Date` でも、JSONでは日時を文字列として送るため、画面側は `string` で受け取ります。

---

## 5. Create: 記録を作成する

### 5-1. 画面からPOSTする

`saveRecord` の核は次の部分です。

```
const response = await fetch("/api/training-records", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ exercise, weight, reps, sets }),
});
```

覚えるポイント:

- `fetch()` はAPIへHTTPリクエストを送る
- `method: "POST"` は新規作成
- `Content-Type` は送信内容がJSONだと伝える
- `JSON.stringify()` はオブジェクトをJSON文字列へ変換する
- `await` は非同期処理が終わるまで待つ

入力値は `input` から来るため、最初は文字列です。

```
const weight = Number(recordDraft.weight);
const reps = Number(recordDraft.reps);
const sets = Number(recordDraft.sets);
```

`Number()` で数値へ変換してからAPIへ送っています。TypeScriptの型は、実行時に文字列を数値へ変換してくれるものではありません。

## 5-2. APIでリクエストを受け取る

```
export const POST = async (request: Request) => {
  const body: unknown = await request.json().catch(() => null);
  const result = createTrainingRecordSchema.safeParse(body);
};
```

`body` を `unknown` にする理由は、ブラウザから何が送られてくるか信用できないためです。`unknown` のままではプロパティを利用できません。Zodの検証に成功して初めて、安全な `result.data` として使えます。

```
if (!result.success) {
  return NextResponse.json(
    { error: "記録内容が正しくありません" },
    { status: 400 },
  );
}

const { exercise, weight, reps, sets, memo } = result.data;
```

これはTypeScriptで重要な「型の絞り込み」です。`result.success` を確認した後では、TypeScriptが `result.data` の存在と型を理解します。

### 5-3. Zodで入力を検証する

```
export const createTrainingRecordSchema = z.object({
  exercise: z.string().trim().min(1).max(100),
  weight: z.number().positive().max(2000),
  reps: z.number().int().min(1).max(1000),
  sets: z.number().int().min(1).max(20),
  memo: z.string().trim().max(500).optional(),
});
```

主な指定の意味:

- `string()`：文字列だけ許可
- `number()`：数値だけ許可
- `positive()`：0より大きい
- `int()`：整数だけ許可
- `min()` / `max()`：範囲を制限
- `optional()`：項目自体がなくてもよい
- `nullable()`：`null`を許可

フロント側のチェックは操作性のため、API側のチェックは安全性のために必要です。フロント側だけを信用してはいけません。

### 5-4. Prismaで保存する

```
await transaction.trainingRecord.createMany({
  data: Array.from({ length: sets }, (_, index) => ({
    userId: user.id,
    reportId: report.id,
    exerciseId: exerciseRecord.id,
    weight,
    reps,
    setOrder: existingSetCount + index + 1,
    memo: memo || null,
    recordedAt: new Date(),
  })),
});
```

`Array.from({ length: sets })`でセット数分のデータを作っています。3セットなら3件の`TrainingRecord`が保存されます。

### 5-5. transactionが必要な理由

新規作成では以下をまとめて実行しています。

1. プロフィールを確認・作成
2. 種目を確認・作成
3. Reportを作成
4. TrainingRecordを作成

```
const records = await prisma.$transaction(async (transaction) => {
  // 複数のDB操作
});
```

途中で1つ失敗した場合、すべてを取り消して不完全なデータが残るのを防ぎます。これをトランザクションと呼びます。

---

## 6. Read: 本日の記録を取得する

### 6-1. 画面からGETする

```
const response = await fetch(
  `/api/training-records?exercise=${encodeURIComponent(targetExercise)}`,
);
```

GETは`method`を省略できます。`?exercise=...`はクエリパラメータです。`encodeURIComponent()`で日本語や記号をURL用に安全な形へ変換します。

### 6-2. APIでクエリパラメータを読む

```
const exercise = new URL(request.url).searchParams.get("exercise")?.trim();
```

`?.`はオプショナルチェーンです。値が`null`なら、その先の`trim()`を実行せず`undefined`になります。

### 6-3. Prismaで一覧を取得する

```
const records = await prisma.trainingRecord.findMany({
  where: {
    userId: user.id,
    recordedAt: { gte: start, lt: end },
    ...(exercise ? { exercise: { name: exercise } } : {}),
  },
  include: { exercise: { select: { name: true } } },
  orderBy: [{ recordedAt: "asc" }, { setOrder: "asc" }],
});
```

必ず説明できるようにしたい部分:

- `where`：検索条件
- `userId: user.id`：本人のデータだけに限定
- `gte`：以上、`lt`：未満
- `include`：関連するExerciseも一緒に取得
- `select`：必要な`name`だけ取得
- `orderBy`：並び順
- `...`：スプレッド構文で条件を必要な場合だけ追加

### 6-4. useEffectで初回取得する

```
useEffect(() => {
  let ignore = false;

  void fetchRecords(exercise).then((nextRecords) => {
    if (!ignore) setRecords(nextRecords);
  });

  return () => {
    ignore = true;
  };
}, [exercise, fetchRecords]);
```

`useEffect`は画面表示後や依存値の変更後に副作用を実行します。ここでは種目が変わるたびに記録を再取得します。

`return`の関数はクリーンアップです。画面を離れた後や次のEffectへ切り替わった後に、古い通信結果でstateを更新しないようにしています。

## 7. Update: 記録を更新する

### 7-1. ID付きURLへPATCHする

```
const response = await fetch(`/api/training-records/${editingRecordId}`, {
  method: "PATCH",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ weight, reps }),
});
```

`PATCH`はデータの一部を変更するときに使います。`editingRecordId`によって、どの記録を更新するかを指定します。

### 7-2. 動的ルートからIDを取得する

```
type TrainingRecordRouteContext = {
  params: Promise<{ id: string }>;
};

const { id } = await context.params;
```

`{ id: string }`は「`id`という文字列プロパティを持つオブジェクト」という型です。Next.js 16では動的ルートの`params`をPromiseとして受け取ります。

### 7-3. 所有者を確認してから更新する

```
const record = await prisma.trainingRecord.findFirst({
  where: { id, userId: user.id },
});
```

`id`だけで検索してはいけません。別ユーザーのIDが分かった場合に更新できてしまうためです。`userId: user.id`を同時に条件へ入れることが、認可の重要ポイントです。

```
const updated = await prisma.trainingRecord.update({
  where: { id: record.id },
  data: result.data,
});
```

Zodで検証済みの`result.data`だけを`data`へ渡しています。

---

# 8. Delete: 記録を削除する

## 8-1. 画面からDELETEする

```
const response = await fetch(`/api/training-records/${recordId}`, {
  method: "DELETE",
});
```

現在の仕様ではカジュアルに操作できるよう、確認ダイアログを挟まず削除します。成功後は一覧を再取得し、トーストを表示します。

## 8-2. Prismaで本人の記録だけ削除する

```
const result = await prisma.trainingRecord.deleteMany({
  where: { id, userId: user.id },
});
```

ここで`delete()`ではなく`deleteMany()`を使う利点は、`id`と`userId`をまとめて条件にでき、対象がなくても例外ではなく`count: 0`を受け取れることです。

```
if (result.count === 0) {
  return NextResponse.json(
    { error: "記録が見つかりません" },
    { status: 404 },
  );
}

return new NextResponse(null, { status: 204 });
```

`204 No Content`は「処理は成功したが、返す本文はない」という意味です。

---

## 9. 認証と認可の違い

この違いは必ず覚えてください。

- 認証（Authentication）:「あなたは誰か」を確認する
- 認可（Authorization）:「そのデータを操作してよいか」を確認する

認証:

```
const supabase = await createClient();
const {
  data: { user },
} = await supabase.auth.getUser();

if (!user) {
  return NextResponse.json({ error: "ログインが必要です" }, { status: 401 });
}
```

認可:

```
where: { id, userId: user.id }
```

ログイン確認だけでは不十分です。DB操作でも`userId`を条件に入れる必要があります。

---

## 10. HTTPステータスコード

| コード | 意味 | RepMateでの使用例 |
| --- | --- | --- |
| `200` | 成功 | 一覧取得・更新 |
| `201` | 作成成功 | 記録の新規作成 |
| `204` | 本文なしで成功 | 削除成功 |
| `400` | リクエスト不正 | Zod検証失敗 |
| `401` | 未認証 | 未ログイン |
| `404` | 対象なし | 記録が存在しない、または本人のものではない |
| `500` | サーバー内部エラー | DB保存失敗 |

`response.ok`はステータスが200番台かどうかを表します。

```
if (!response.ok) {
  throw new Error(data.error ?? "記録を保存できませんでした");
}
```

`??`はNull合体演算子です。左側が`null`または`undefined`のときだけ右側を使います。

---

## 11. try / catch / finally

```
setIsSavingRecord(true);

try {
  // API通信
} catch (error) {
  setToastMessage(
    error instanceof Error
      ? error.message
      : "記録を保存できませんでした",
  );
} finally {
  setIsSavingRecord(false);
}
```

- `try`：成功する可能性のある処理
- `catch`：通信やDB操作が失敗した場合
- `finally`：成功・失敗に関係なく最後に実行
- `error instanceof Error`：`error`がErrorオブジェクトか型を絞り込む

保存中フラグを`finally`で戻すことで、エラー時にもボタンが永遠に無効になるのを防ぎます。

---

## 12. React Hooksで覚える箇所

### useState

画面内で変化する値を保持します。

```
const [records, setRecords] = useState<TrainingRecordItem[]>([]);
const [editingRecordId, setEditingRecordId] = useState<string | null>(null);
```

`TrainingRecordItem[]`は「TrainingRecordItemの配列」です。`string | null`は文字列またはnullを許可するUnion型です。

### useEffect

画面表示後の取得や、種目変更後の再取得に使います。

### useCallback

関数を再利用し、依存している`useEffect`が不要に再実行されるのを抑えます。

```
const fetchRecords = useCallback(async (targetExercise: Exercise) => {
  // 取得処理
}, []);
```

### useMemo

依存値が変わったときだけ計算し直します。

```
const apiMessages = useMemo(
  () => messages.map(({ role, content }) => ({ role, content })),
  [messages],
);
```

CRUDの理解には、まず`useState`と`useEffect`を優先してください。`useCallback`と`useMemo`は、その後で十分です。

---

## 13. TypeScriptで必須の構文

### オブジェクト型

```
type Item = {
  id: string;
  weight: number;
};
```

### Union型

```
type Status = "idle" | "loading" | "success" | "error";
```

決められた文字列だけを許可できます。

### 配列型

```
const records: TrainingRecordItem[] = [];
```

### keyof

```
field: keyof TrainingRecordDraft
```

`TrainingRecordDraft`のキーである`weight | reps | sets`だけを許可します。存在しないキーの入力ミスをコンパイル時に防げます。

### ジェネリクス

```
useState<TrainingRecordItem[]>([])
```

`<TrainingRecordItem[]>`によって、stateへ入る値の型を指定しています。

### 型アサーション

```
const data = (await response.json()) as { error?: string };
```

`as`は「この型として扱う」とTypeScriptへ伝える構文ですが、実行時検証はしません。外部入力を本当に安全にするにはZodなどが必要です。

### 分割代入

```
const { exercise, weight, reps, sets, memo } = result.data;
```

オブジェクトから必要なプロパティを取り出します。

---

## 14. 画面更新が2種類ある理由

```
await loadRecords(exercise);
router.refresh();
```

- `loadRecords(exercise)`：ReportScreen内の本日の記録を即時更新
- `router.refresh()`：Server Componentを再取得し、ホームや進捗に最新DB情報を反映

Reactのstateと、Next.jsのServer Componentが持つデータは別物です。この違いを理解すると「DBは更新されたのに画面が変わらない」という問題を追いやすくなります。

---

## 15. 目標機能のCRUDはどうなっているか

目標機能はトレーニング記録とは少し異なります。

| 操作 | 実装 |
| --- | --- |
| Create | 選択した目標を`training_goals`へ保存 |
| Read | 現在のactive目標や保存済み候補を取得 |
| Update | 保存済み候補から目標を変更 |
| Delete相当 | 物理削除せず`status: "cancelled"`へ変更 |

DBから行そのものを消すことを物理削除、状態を変更して利用対象から外すことを論理削除と呼びます。履歴を残したい目標には論理削除が向いています。

一方、トレーニング記録の削除は現在`DELETE`による物理削除です。

---

## 16. 最低限、自力で書けるようにしたいテンプレート

### クライアント側

```
const createItem = async () => {
  try {
    const response = await fetch("/api/items", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ name: "sample" }),
    });

    const data = await response.json();
    if (!response.ok) throw new Error(data.error);

    // 一覧を再取得する、またはstateへ追加する
  } catch (error) {
    console.error(error);
  }
};
```

### API側

```
export const POST = async (request: Request) => {
  const body: unknown = await request.json().catch(() => null);
  const result = schema.safeParse(body);

  if (!result.success) {
    return NextResponse.json({ error: "入力エラー" }, { status: 400 });
  }

  const item = await prisma.item.create({
    data: result.data,
  });

  return NextResponse.json({ item }, { status: 201 });
};
```

丸暗記するより、次の順番を覚えてください。

```
受け取る → 検証する → DB操作する → 結果を返す
```

---

## 17. セキュリティ上、外してはいけない記述

次は「短くしたいから」と削除してはいけません。

```
if (!user) {
  return NextResponse.json({ error: "ログインが必要です" }, { status: 401 });
}
```

```
const result = schema.safeParse(body);
if (!result.success) {
  // 400を返す
}
```

```
where: { id, userId: user.id }
```

認証・入力検証・所有者確認の3つが基本セットです。

---

## 18. 学習課題

### レベル1: 読んで答える

1. `POST` と `PATCH` の違いは何ですか？
2. なぜ `request.json()` の結果を `unknown` にしていますか？
3. `response.ok` が `false` になるのはどんな場合ですか？
4. `userId: user.id` を検索条件へ入れる理由は何ですか？
5. `finally` がない場合、どんな不具合が起きますか？

### レベル2: 小さく変更する

1. 記録編集で `memo` も変更できるようにする

2. 削除中だけ削除ボタンを無効化する
3. Zodの重量上限を変更し、なぜその値にしたか説明する
4. APIエラー時のメッセージをステータスごとに変える

### レベル3: 自力で追加する

1. 記録詳細を1件取得する `GET /api/training-records/[id]`
2. 今日ではなく日付を指定して取得する機能
3. 直前の削除を取り消せる論理削除方式

---

## 19. 理解度チェックリスト

- [ ] CRUDの4単語とHTTPメソッドを対応させられる
- [ ] `fetch` のPOSTを見ずに書ける
- [ ] `async / await` を説明できる
- [ ] `unknown` と `any` の違いを説明できる
- [ ] `safeParse()` 成功後に `result.data` を使う理由が分かる
- [ ] `findMany` 、 `createMany` 、 `update` 、 `deleteMany` の役割が分かる
- [ ] 認証と認可の違いを説明できる
- [ ] `userId: user.id` が必要な理由を説明できる
- [ ] `try / catch / finally` を書ける
- [ ] `useState` と `useEffect` の役割を説明できる
- [ ] `string | null` 、配列型、 `keyof` を読める
- [ ] DB更新後に画面を更新する方法を説明できる

8個以上説明できれば、今回のCRUD実装の主要部分はかなり身についています。説明できない項目は、その項目に関係する実コードへ戻って確認してください。

---

## 20. 最後に押さえる要点

RepMateのCRUDで最も大切なのは、Prismaのメソッド名を暗記することだけではありません。

1. 画面が正しいHTTPリクエストを作る
2. APIがユーザーを認証する

3. APIが外部入力をZodで検証する
4. Prismaが本人のデータだけを操作する
5. 成功・失敗を適切なステータスで返す
6. 画面が結果を受けてstateを更新する

この一連の流れを、`Create → Read → Update → Delete`の順に実コードを指しながら説明できれば、次のCRUD機能は自力で組み立てやすくなります。

---